SEGUNDO PARCIAL · PRÁCTICO

MediControl

Sistema monolítico modular para la gestión de médicos y citas médicas

Flask · SQLAlchemy · Flask-Migrate · Bootstrap 5 · SQLite

ESTUDIANTE
Mayte Mikaela Quispe Perez

DOCENTE
M. Sc. Mario Torrez C.

UNIVERSIDAD
Universidad Pública de El Alto

MATERIA
TEM-742 Tecnologías Emergentes II

EVALUACIÓN
Segundo Parcial — Práctico

FECHA
3 de noviembre de 2026

Repositorio GitHub:
`https://github.com/mikaelamay/segundo_parcial_mmqp`

El Alto — Bolivia

CONTENIDO

Índice del informe

Documentación del desarrollo, configuración, modelos, migraciones, CRUDs, plantillas y pruebas de funcionamiento del sistema MediControl.

1 Introducción y objetivos 3

2 Verificación de requerimientos 4

3 Parte 1 — Arquitectura y configuración 5

4 Parte 2 — Modelos y migraciones 10

5 Parte 3 — CRUD y plantillas 13

6 Parte 4 — Funcionamiento de la aplicación 23

7 Conclusiones 34

8 Anexo — Ejecución y entregables 35

Resumen del proyecto. MediControl es una aplicación web desarrollada con Flask que implementa el patrón *Application Factory*, tres Blueprints independientes, modelos relacionados con SQLAlchemy, migraciones versionadas con Flask-Migrate y una interfaz responsiva construida sobre Bootstrap 5. La base de datos utilizada es SQLite, tal como solicita el enunciado del examen.

SECCIÓN 1

Introducción y objetivos

La empresa **MediControl** necesita administrar la información de sus médicos, sus pacientes y las citas que se agendan diariamente. Para resolver esta necesidad se construyó un **sistema monolítico modular**: una sola aplicación desplegable, pero organizada internamente por módulos de negocio independientes (médicos, pacientes y citas), cada uno con sus propias rutas y plantillas.

El backend se implementó con **Flask**, el acceso a datos con **SQLAlchemy**, el control de versiones del esquema con **Flask-Migrate** y la interfaz con **Bootstrap 5**. La base de datos es **SQLite**, almacenada en el archivo `instance/medicontrol.db`, conforme al requerimiento de la Parte 1 del examen.

Objetivos

- Implementar el patrón **Application Factory** mediante `create_app()`.
- Crear y registrar los Blueprints `bp_medicos`, `bp_pacientes` y `bp_citas` con sus prefijos de URL.
- Definir los modelos y sus relaciones uno a muchos, y versionar el esquema con migraciones.
- Construir los CRUD completos y las plantillas que heredan de una plantilla base común.
- Verificar el funcionamiento con datos de prueba y operaciones reales (alta, edición, consulta y baja).

Alcance y decisiones de diseño

El sistema cubre el ciclo completo de gestión: registro de médicos y pacientes, agenda de citas, edición y eliminación controlada. Se incorporaron validaciones adicionales para evitar citas duplicadas de un mismo médico en la misma fecha y hora, y para impedir la eliminación de médicos o pacientes que tengan citas asociadas, protegiendo así la integridad referencial.

La interfaz incluye un panel principal con indicadores, búsqueda de médicos y pacientes, filtros de citas por fecha y médico, mensajes de confirmación y alertas de error. Todos los recursos de Bootstrap se guardaron localmente en `app/static/vendor`, por lo que la aplicación funciona sin depender de una conexión a Internet.

> **Organización del documento.** Las secciones 3 a 6 corresponden exactamente a las cuatro partes del examen. Cada una incluye una explicación breve y las capturas de pantalla que evidencian su implementación.

SECCIÓN 2

Verificación de requerimientos

La siguiente tabla relaciona cada requerimiento del enunciado con su evidencia dentro del proyecto y su estado de cumplimiento.

N.º	REQUERIMIENTO	EVIDENCIA EN EL PROYECTO	ESTADO
1	Patrón Flask Application Factory	`app/__init__.py` → `create_app()`	**Cumplido**
2	Blueprints `bp_medicos`, `bp_pacientes` y `bp_citas`	`app/blueprints/` con sus archivos `routes.py`	**Cumplido**
3	Prefijos `/medicos`, `/pacientes` y `/citas`	Registro con `url_prefix` en la fábrica	**Cumplido**
4	Base de datos SQLite	`config.py` → `sqlite:///…/medicontrol.db`	**Cumplido**
5	Modelos Médico, Paciente y Cita con relaciones	`app/models/` + `relationship()` y claves foráneas	**Cumplido**
6	Inicializar, crear y aplicar migraciones	Flask-Migrate · revisión `2a61b8b3acee`	**Cumplido**
7	CRUD de médicos, pacientes y citas	Rutas y plantillas para listar, crear, editar y eliminar	**Cumplido**
8	Plantilla base y plantillas heredadas	`base.html` + `{% extends "base.html" %}`	**Cumplido**
9	Datos de prueba y operaciones principales	`seed.py` y capturas de la Parte 4	**Cumplido**
10	URL del repositorio en GitHub	Portada, anexo y `README.md`	**Cumplido**

Resultado técnico

22 capturas de pantalla, 3 módulos CRUD, 10 citas de prueba, 6 médicos y 6 pacientes registrados. Todas las rutas responden correctamente y las validaciones fueron comprobadas de forma manual y automatizada.

Resultado académico

Se aplicaron los contenidos de la materia: fábrica de aplicaciones, Blueprints, ORM, migraciones, herencia de plantillas y Bootstrap, con una estructura ordenada y fácil de mantener.

SECCIÓN 3 · PARTE 1

Arquitectura y configuración

Application Factory, Blueprints con prefijos y configuración de SQLite.

3.1. Patrón Application Factory

Toda la aplicación se construye dentro de la función `create_app()` ubicada en `app/__init__.py`. Esta función recibe una clase de configuración, crea la instancia de Flask, inicializa las extensiones y registra los Blueprints. Gracias a este patrón, la aplicación puede crearse con configuraciones distintas (por ejemplo, pruebas o producción) sin duplicar código.

app/__init__.py

```
def create_app(config_class=Config):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)

    # Carpeta instance/ para la base SQLite
    os.makedirs(app.instance_path, exist_ok=True)

    # Inicialización de extensiones
    db.init_app(app)
    migrate.init_app(app, db)

    # Registro de modelos para SQLAlchemy y Flask-Migrate
    from app import models

    # Registro de Blueprints con sus prefijos
    from app.blueprints.medicos import bp_medicos
    from app.blueprints.pacientes import bp_pacientes
    from app.blueprints.citas import bp_citas

    app.register_blueprint(bp_medicos, url_prefix="/medicos")
    app.register_blueprint(bp_pacientes, url_prefix="/pacientes")
    app.register_blueprint(bp_citas, url_prefix="/citas")

    return app
```

3.2. Blueprints registrados

Cada módulo de negocio vive en su propio paquete dentro de `app/blueprints/`. El archivo `__init__.py` define el Blueprint y el archivo `routes.py` contiene sus rutas. De este modo, el monolito queda dividido en módulos con bajo acoplamiento.

BLUEPRINT	PREFIJO	ARCHIVO DE RUTAS	RESPONSABILIDAD
`bp_medicos`	`/medicos`	`blueprints/medicos/routes.py`	CRUD de médicos y búsqueda por especialidad
`bp_pacientes`	`/pacientes`	`blueprints/pacientes/routes.py`	CRUD de pacientes y validación de teléfono
`bp_citas`	`/citas`	`blueprints/citas/routes.py`	Agenda de citas, filtros y control de choques de horario

3.3. Configuración de SQLite

La cadena de conexión se define en `config.py`. Flask crea automáticamente la carpeta `instance/` y la base de datos se genera al aplicar las migraciones. También se permite sobrescribir la URL mediante la variable de entorno `DATABASE_URL`.

config.py

```
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "medicontrol-clave-de-desarrollo-2026")

    # El examen solicita SQLite como motor de base de datos.
    SQLALCHEMY_DATABASE_URI = os.environ.get(
        "DATABASE_URL",
        f"sqlite:///{BASE_DIR / 'instance' / 'medicontrol.db'}",
    )
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_ENGINE_OPTIONS = {"pool_pre_ping": True}
```

3.4. Estructura del proyecto

La organización separa configuración, modelos, Blueprints, plantillas y recursos estáticos. Las migraciones y el script de datos de prueba se mantienen en la raíz para facilitar la puesta en marcha.

Estructura del proyecto — mayte@debian: ~/examen_2doparcialemergentes

```
$ tree -I "venv|__pycache__|*.pyc|docs|.git" --dirsfirst
.
├── app
│   ├── blueprints
│   │   ├── citas
│   │   │   ├── __init__.py
│   │   │   └── routes.py
│   │   ├── medicos
│   │   │   ├── __init__.py
│   │   │   └── routes.py
│   │   ├── pacientes
│   │   │   ├── __init__.py
│   │   │   └── routes.py
│   │   └── __init__.py
│   ├── models
│   │   ├── cita.py
│   │   ├── __init__.py
│   │   ├── medico.py
│   │   └── paciente.py
│   ├── static
│   │   ├── css
│   │   │   └── styles.css
│   │   ├── img
│   │   │   └── favicon.svg
│   │   └── vendor
│   │       ├── bootstrap
│   │       │   ├── bootstrap.bundle.min.js
│   │       │   └── bootstrap.min.css
│   │       └── bootstrap-icons
│   │           ├── fonts
│   │           │   ├── bootstrap-icons.woff
│   │           │   └── bootstrap-icons.woff2
│   │           └── bootstrap-icons.min.css
│   ├── templates
│   │   ├── citas
│   │   │   ├── form.html
│   │   │   └── index.html
│   │   ├── medicos
│   │   │   ├── form.html
│   │   │   └── index.html
│   │   ├── pacientes
│   │   │   ├── form.html
│   │   │   └── index.html
│   │   ├── 404.html
│   │   ├── base.html
│   │   └── index.html
│   ├── extensions.py
│   └── __init__.py
├── instance
│   └── medicontrol.db
├── migrations
│   ├── versions
│   │   └── 2a61b8b3acee_esquema_inicial_medicos_pacientes_y_.py
│   ├── alembic.ini
│   ├── env.py
│   ├── README
│   └── script.py.mako
├── config.py
├── requirements.txt
├── run.py
└── seed.py

21 directories, 39 files
```

Figura 1. Estructura de directorios del proyecto, generada con el comando `tree`. Se aprecian los tres Blueprints, los modelos, las plantillas, las migraciones y la carpeta `instance/` con la base SQLite.

3.5. Aplicación en ejecución

Al iniciar el servidor con `flask --app run.py run`, la ruta principal `/` presenta el panel de control. En él se muestran los indicadores de médicos, pacientes y citas, las próximas citas ordenadas por fecha y hora, y accesos rápidos a las operaciones más frecuentes.

La interfaz es responsiva y utiliza una barra de navegación con acceso directo a los tres módulos. Los colores y componentes fueron personalizados sobre Bootstrap 5 para darle una identidad visual coherente con el sector salud.

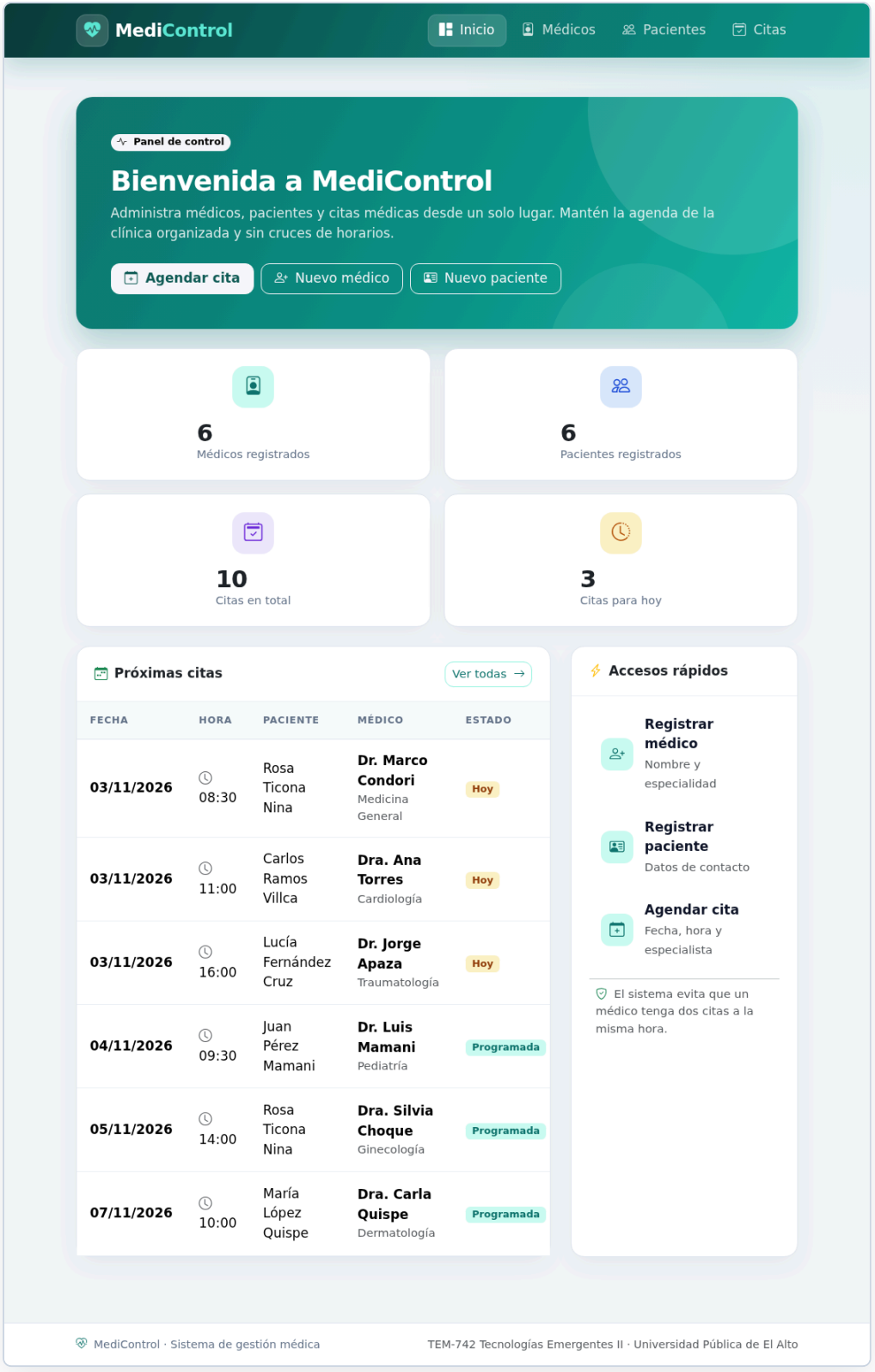


Figura 2. Panel principal de MediControl en ejecución. Se visualizan los contadores (6 médicos, 6 pacientes, 10 citas), las citas de hoy y las próximas citas agendadas, además de los accesos rápidos.

Resultado de la Parte 1. La aplicación se crea mediante `create_app()`, los tres Blueprints están registrados con los prefijos solicitados y SQLite quedó configurado como motor de persistencia. La estructura modular permite agregar nuevos módulos (por ejemplo, especialidades o historiales) sin alterar los existentes.

SECCIÓN 4 · PARTE 2

Modelos y migraciones

Entidades Médico, Paciente y Cita, relaciones uno a muchos y versionado del esquema.

4.1. Modelos implementados

Los modelos se organizaron en un paquete (`app/models/`) con un archivo por entidad. Todos heredan de `db.Model` y declaran explícitamente el nombre de su tabla.

MODELO	TABLA	CAMPOS	RELACIÓN
Medico	`medicos`	id, nombre, especialidad	Un médico tiene muchas citas
Paciente	`pacientes`	id, nombre, telefono	Un paciente tiene muchas citas
Cita	`citas`	id, fecha, hora, medico_id, paciente_id	Pertenece a un médico y a un paciente

app/models/medico.py · paciente.py · cita.py

```
class Medico(db.Model):
    __tablename__ = "medicos"
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(120), nullable=False)
    especialidad = db.Column(db.String(120), nullable=False)
    citas = db.relationship("Cita", back_populates="medico")

class Paciente(db.Model):
    __tablename__ = "pacientes"
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(120), nullable=False)
    telefono = db.Column(db.String(30), nullable=False)
    citas = db.relationship("Cita", back_populates="paciente")

class Cita(db.Model):
    __tablename__ = "citas"
    id = db.Column(db.Integer, primary_key=True)
    fecha = db.Column(db.Date, nullable=False)
    hora = db.Column(db.Time, nullable=False)
    medico_id = db.Column(db.Integer, db.ForeignKey("medicos.id"), nullable=False)
    paciente_id = db.Column(db.Integer, db.ForeignKey("pacientes.id"), nullable=False)
    medico = db.relationship("Medico", back_populates="citas")
    paciente = db.relationship("Paciente", back_populates="citas")
```

4.2. Relaciones e integridad

La relación se declara en ambos sentidos con `back_populates`: desde **Medico** y **Paciente** se accede a la colección `citas`, y desde **Cita** se accede al `medico` y al `paciente`. Las claves foráneas `medico_id` y `paciente_id` son obligatorias y cuentan con índices para acelerar las consultas.

Además de la integridad declarada en la base de datos, la capa de aplicación impide eliminar un médico o un paciente que tenga citas asociadas, y valida que no exista otra cita para el mismo médico en la misma fecha y

hora.

4.3. Migraciones con Flask-Migrate

Se inicializó el repositorio de migraciones, se generó la migración inicial a partir de los modelos y se aplicó sobre SQLite. La revisión generada es `2a61b8b3acee`. Los comandos utilizados fueron:

Terminal — inicialización y aplicación de migraciones

```
# 1) Inicializar el repositorio de migraciones
flask --app run.py db init

# 2) Generar la migración inicial comparando los modelos con la BD
flask --app run.py db migrate -m "Esquema inicial: medicos, pacientes y citas"

# 3) Aplicar los cambios a la base de datos
flask --app run.py db upgrade

# 4) Verificar la revisión activa
flask --app run.py db current
```

Para comprobar que las migraciones funcionan desde cero, se aplicó la misma revisión sobre una base temporal (`/tmp/medicontrol_demo.db`). La salida confirma la creación de las tablas `medicos`, `pacientes`, `citas` y `alembic_version`.

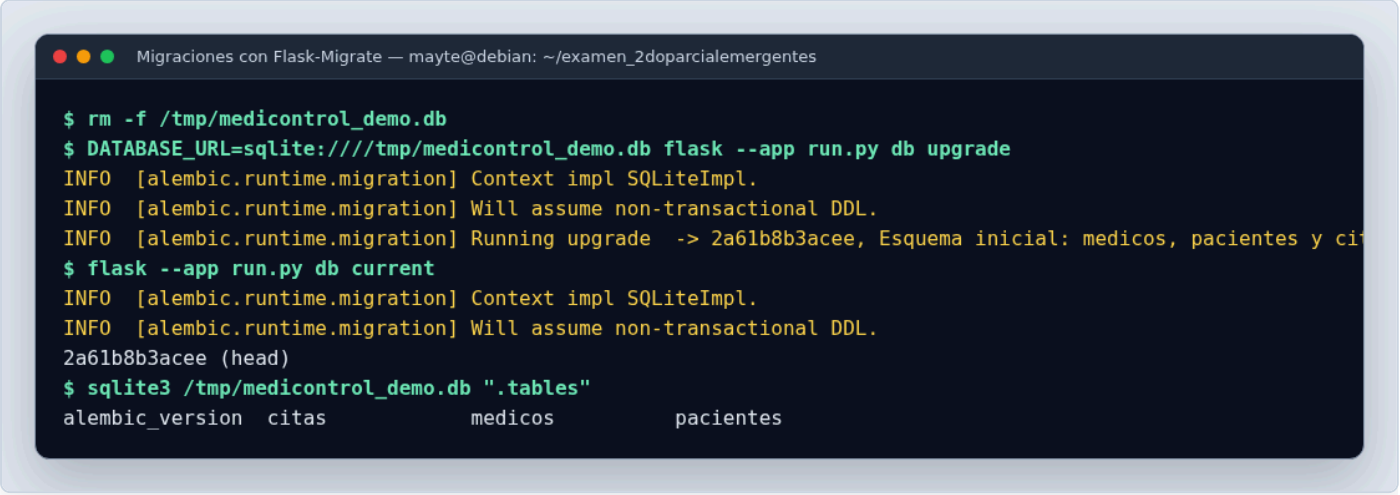


Figura 3. Aplicación de la migración inicial sobre una base SQLite nueva. Flask-Migrate ejecuta la revisión `2a61b8b3acee` y luego se verifica la revisión activa con `db current`.

4.4. Esquema resultante en SQLite

El esquema generado incluye las tres tablas de negocio, sus claves primarias, las claves foráneas de `citas` y los índices correspondientes. La tabla `alembic_version` es administrada por Flask-Migrate para registrar la revisión aplicada.

Esquema de la base de datos SQLite — mayte@debian: ~/examen_2doparcialemergentes

```
$ sqlite3 instance/medicontrol.db ".schema medicos"
CREATE TABLE medicos (
        id INTEGER NOT NULL,
        nombre VARCHAR(120) NOT NULL,
        especialidad VARCHAR(120) NOT NULL,
        PRIMARY KEY (id)
);
$ sqlite3 instance/medicontrol.db ".schema pacientes"
CREATE TABLE pacientes (
        id INTEGER NOT NULL,
        nombre VARCHAR(120) NOT NULL,
        telefono VARCHAR(30) NOT NULL,
        PRIMARY KEY (id)
);
$ sqlite3 instance/medicontrol.db ".schema citas"
CREATE TABLE citas (
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
        hora TIME NOT NULL,
        medico_id INTEGER NOT NULL,
        paciente_id INTEGER NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(medico_id) REFERENCES medicos (id),
        FOREIGN KEY(paciente_id) REFERENCES pacientes (id)
);
CREATE INDEX ix_citas_medico_id ON citas (medico_id);
CREATE INDEX ix_citas_paciente_id ON citas (paciente_id);
```

Figura 4. Esquema físico de la base de datos obtenido con `sqlite3 .schema`. Se observan las claves foráneas `medico_id` y `paciente_id` y los índices `ix_citas_medico_id` e `ix_citas_paciente_id`.

Resultado de la Parte 2. Los tres modelos fueron creados con sus relaciones uno a muchos, la migración inicial quedó versionada en el repositorio y el esquema fue aplicado correctamente en SQLite. La base puede reconstruirse desde cero ejecutando `flask db upgrade`.

SECCIÓN 5 · PARTE 3

CRUD y plantillas

Operaciones de listado, creación, edición y eliminación para los tres módulos, con una plantilla base común.

5.1. Rutas implementadas

Cada Blueprint expone un CRUD completo. Las operaciones de eliminación se realizan mediante `POST` y solicitan confirmación en el navegador; las demás usan `GET` para mostrar formularios y `POST` para procesarlos.

MÓDULO	MÉTODO Y RUTA	FUNCIÓN	DESCRIPCIÓN
Médicos	`GET /medicos/`	index	Listado y búsqueda
	`GET/POST /medicos/nuevo`	crear	Registro de médico
	`GET/POST /medicos/<id>/editar`	editar	Actualización
	`POST /medicos/<id>/eliminar`	eliminar	Baja controlada
Pacientes	`GET /pacientes/`	index	Listado y búsqueda
	`GET/POST /pacientes/nuevo`	crear	Registro de paciente
	`GET/POST /pacientes/<id>/editar`	editar	Actualización
	`POST /pacientes/<id>/eliminar`	eliminar	Baja controlada
Citas	`GET /citas/`	index	Listado y filtros
	`GET/POST /citas/nueva`	crear	Agendamiento
	`GET/POST /citas/<id>/editar`	editar	Reprogramación
	`POST /citas/<id>/eliminar`	eliminar	Cancelación

5.2. Plantilla base y herencia

La plantilla `app/templates/base.html` centraliza el documento HTML, la barra de navegación, los mensajes flash, el pie de página y la carga de Bootstrap. Las demás plantillas heredan de ella mediante `{% extends "base.html" %}` y sobrescriben los bloques `title`, `content` y, cuando es necesario, `scripts`.

app/templates/base.html (fragmento)

```
<nav class="navbar navbar-expand-lg navbar-dark app-navbar">
  <a class="navbar-brand" href="{{ url_for('index') }}">MediControl</a>
  <a href="{{ url_for('bp_medicos.index') }}">Médicos</a>
  <a href="{{ url_for('bp_pacientes.index') }}">Pacientes</a>
  <a href="{{ url_for('bp_citas.index') }}">Citas</a>
</nav>
```

```
<main class="container py-4">
  {% with mensajes = get_flashed_messages(with_categories=true) %}
    {% for categoria, mensaje in mensajes %}
      <div class="alert alert-{{ categoria }}">{{ mensaje }}</div>
    {% endfor %}
  {% endwith %}
  {% block content %}{% endblock %}
</main>
```

Las plantillas de cada módulo se ubican en carpetas separadas (`templates/medicos/`, `templates/pacientes/` y `templates/citas/`). Cada una extiende la base y reutiliza los estilos, la navegación y el sistema de mensajes.

app/templates/medicos/index.html (inicio)

```
{% extends "base.html" %}

{% block title %}Médicos{% endblock %}

{% block content %}
  <h1>Médicos</h1>
  <a href="{{ url_for('bp_medicos.crear') }}">Nuevo médico</a>
  <table>...</table>
{% endblock %}
```

5.3. CRUD de Médicos

El listado muestra el identificador, el nombre, la especialidad y la cantidad de citas de cada médico. Incluye búsqueda por nombre o especialidad y acciones de edición y eliminación. Si el médico tiene citas, el sistema impide la baja y muestra una advertencia.

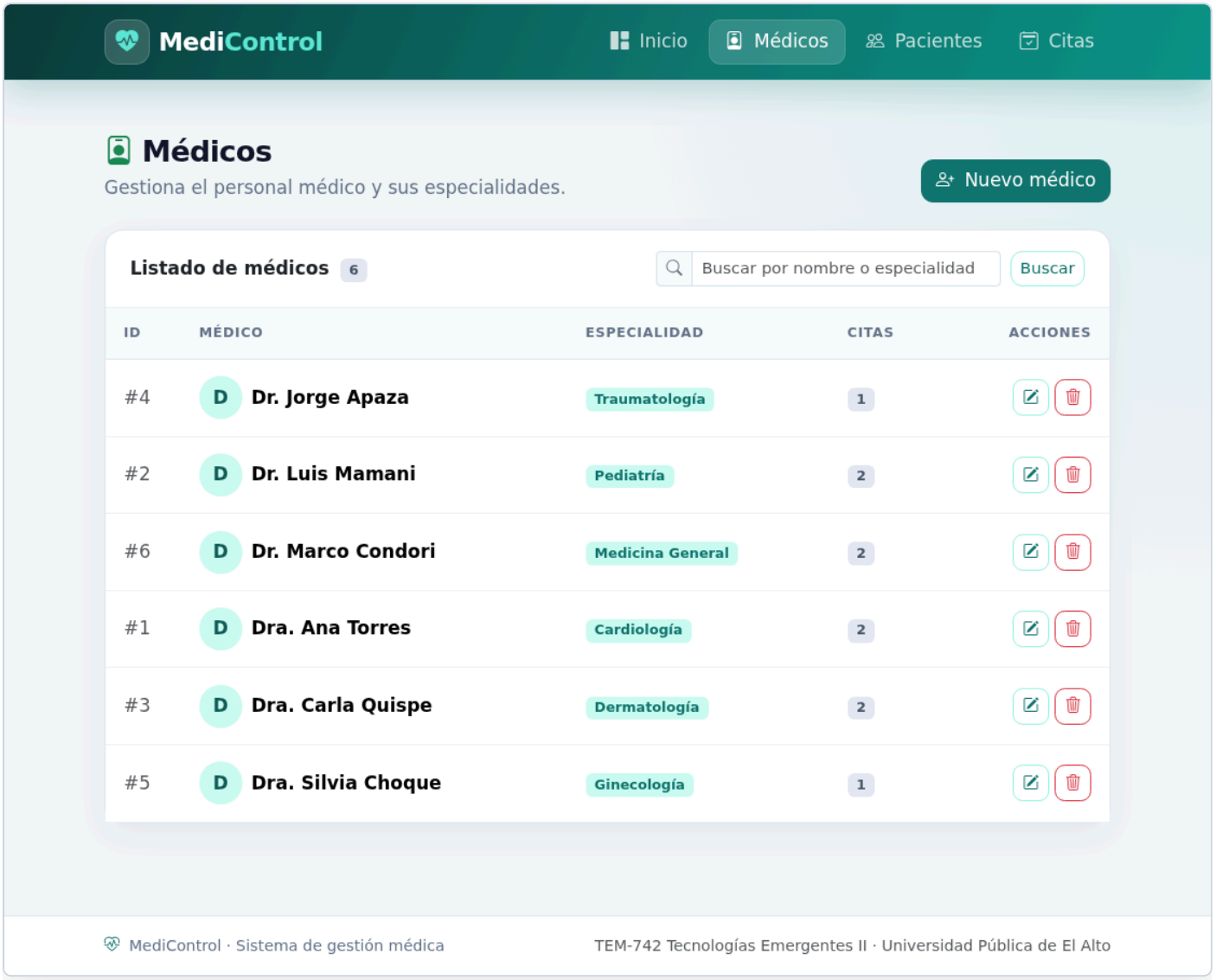


Figura 5. Listado de médicos con 6 registros. Cada fila muestra el avatar, la especialidad y el número de citas; las acciones permiten editar o eliminar.

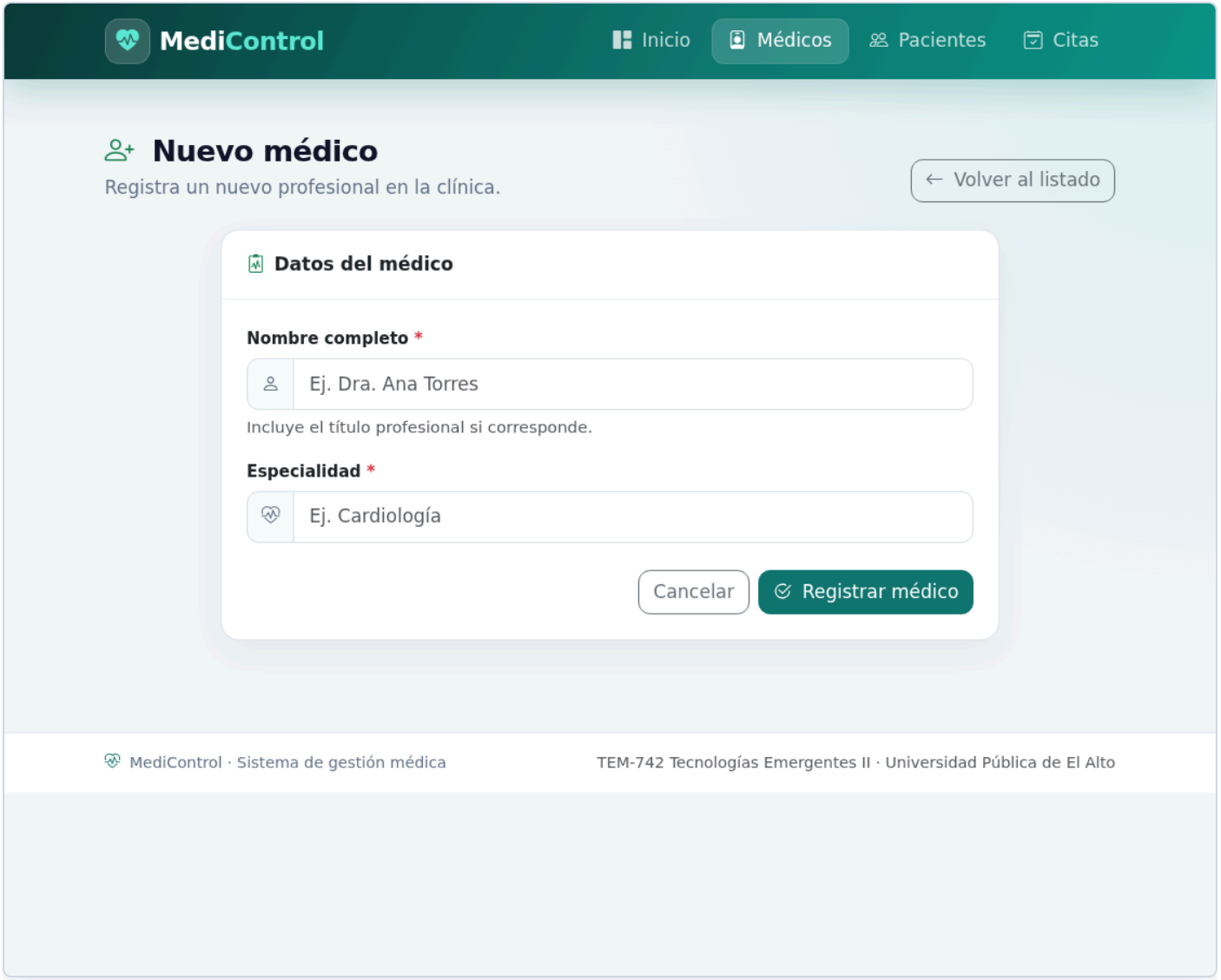


Figura 6. Formulario de registro de un médico. Los campos nombre y especialidad son obligatorios y se validan en el servidor antes de guardar.

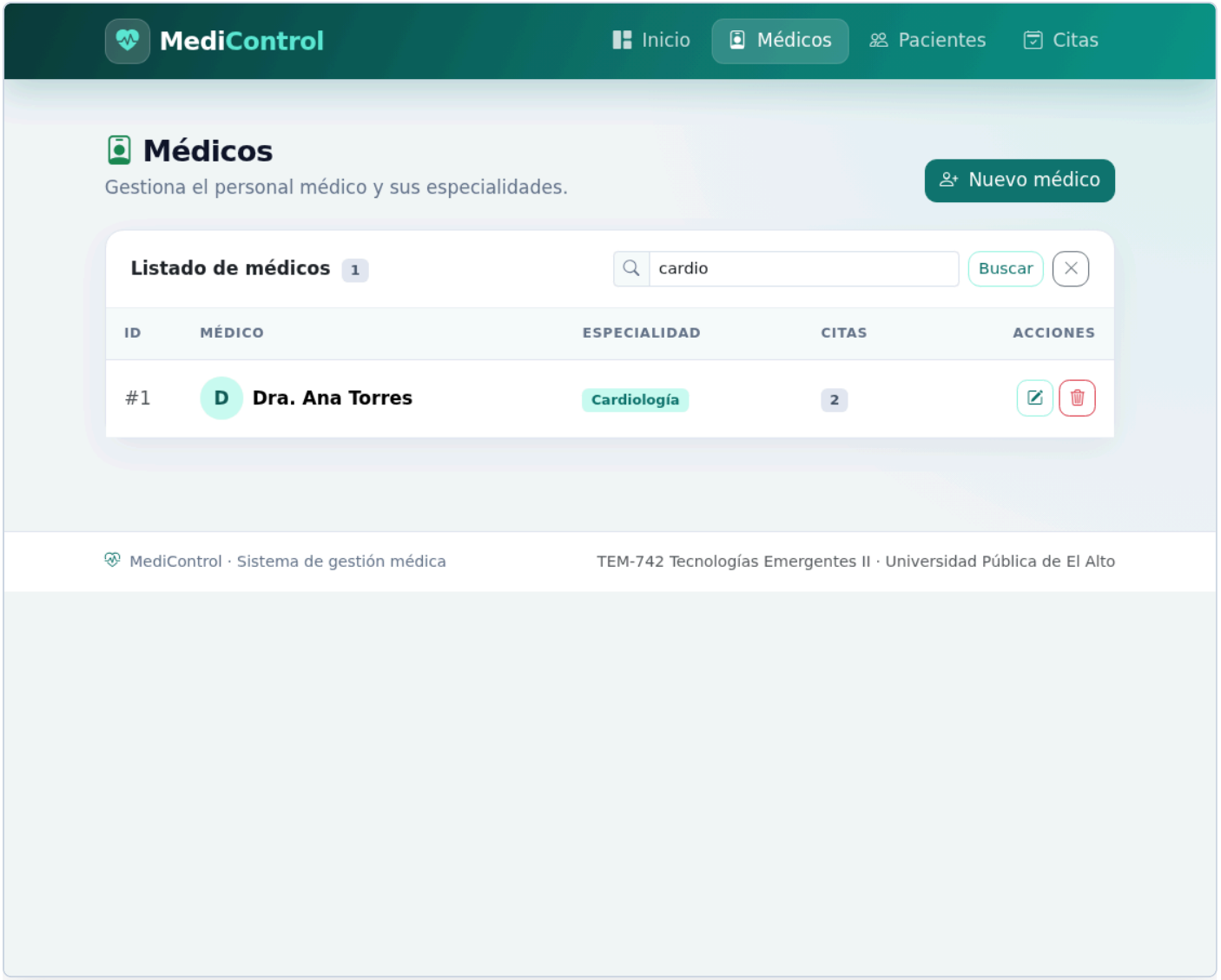


Figura 7. Búsqueda por especialidad. El filtro se aplica sobre el nombre y la especialidad mediante una consulta `ILIKE`.

5.4. CRUD de Pacientes

El módulo de pacientes permite registrar, buscar, editar y eliminar pacientes. El teléfono se valida con una expresión regular que admite números, espacios, paréntesis, guiones y el signo `+`. Al igual que en médicos, no se permite eliminar un paciente con citas registradas.

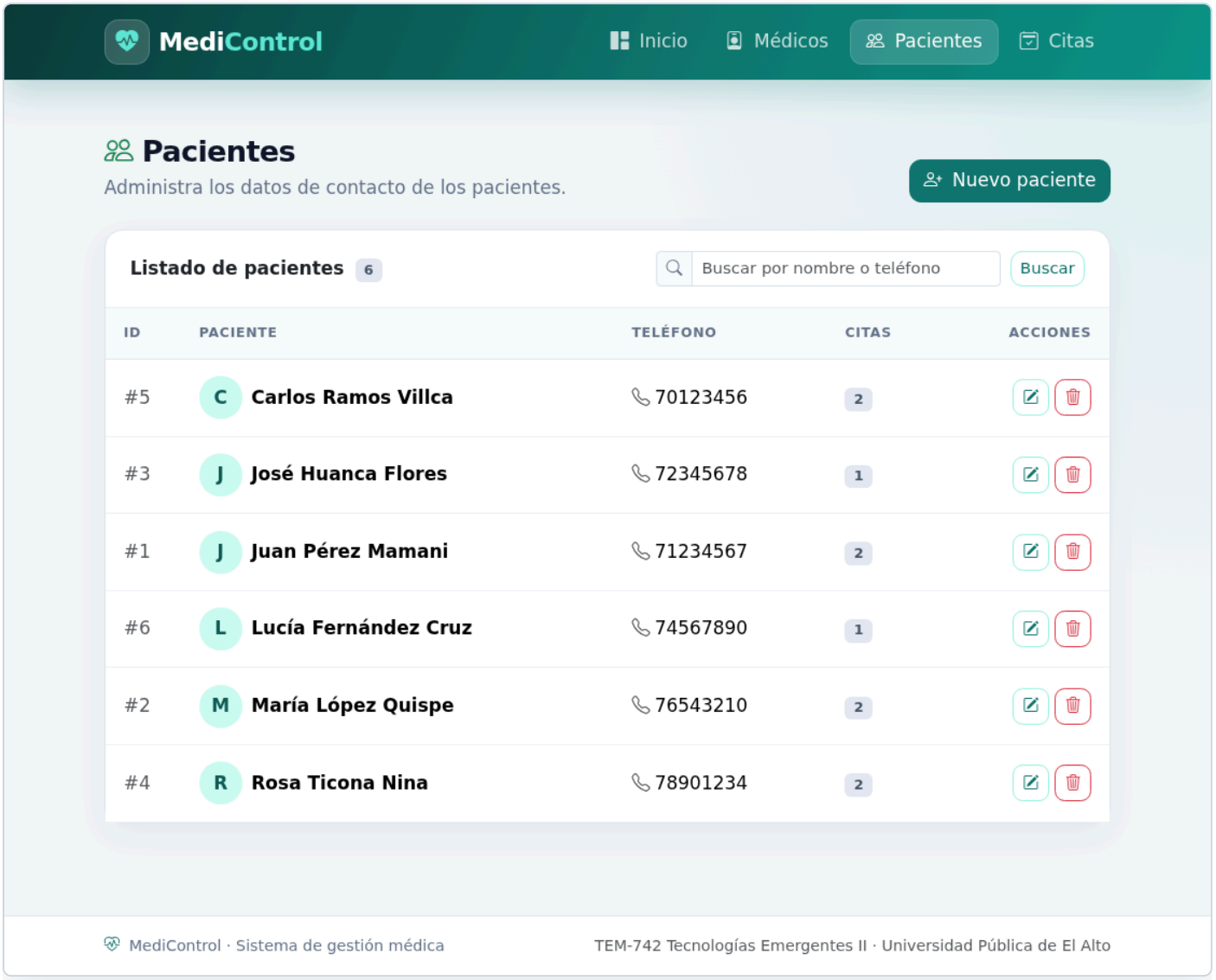


Figura 8. Listado de pacientes con su teléfono y la cantidad de citas asociadas. La búsqueda funciona por nombre o por teléfono.

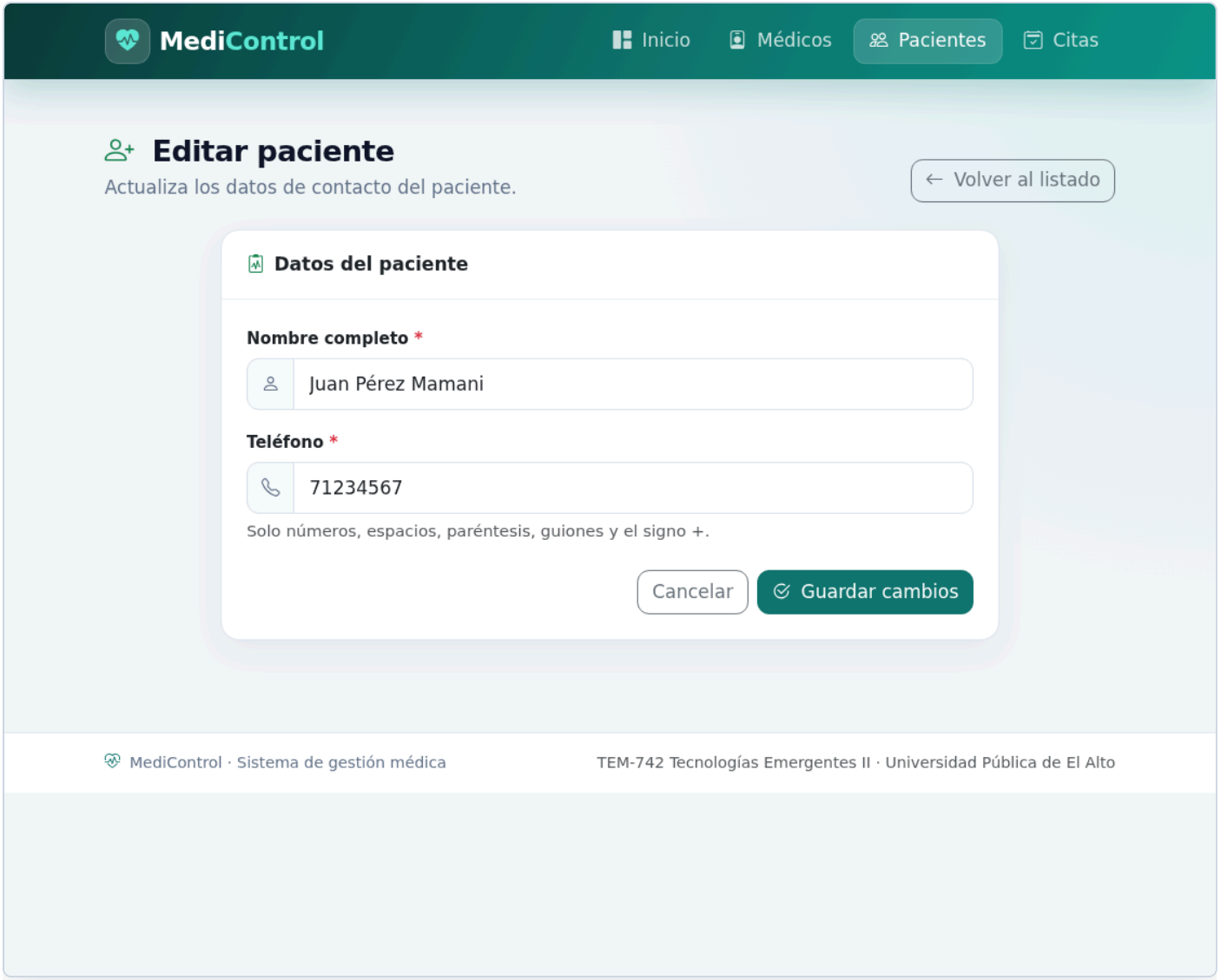


Figura 9. Formulario de edición de un paciente. Los datos existentes se cargan automáticamente y el formulario reutiliza la misma plantilla que el alta.

5.5. CRUD de Citas

El módulo de citas relaciona médicos y pacientes. El listado muestra la fecha, la hora, el médico con su especialidad, el paciente con su teléfono y un estado calculado (*Programada*, *Hoy* o *Atendida*). También permite filtrar por fecha y por médico.

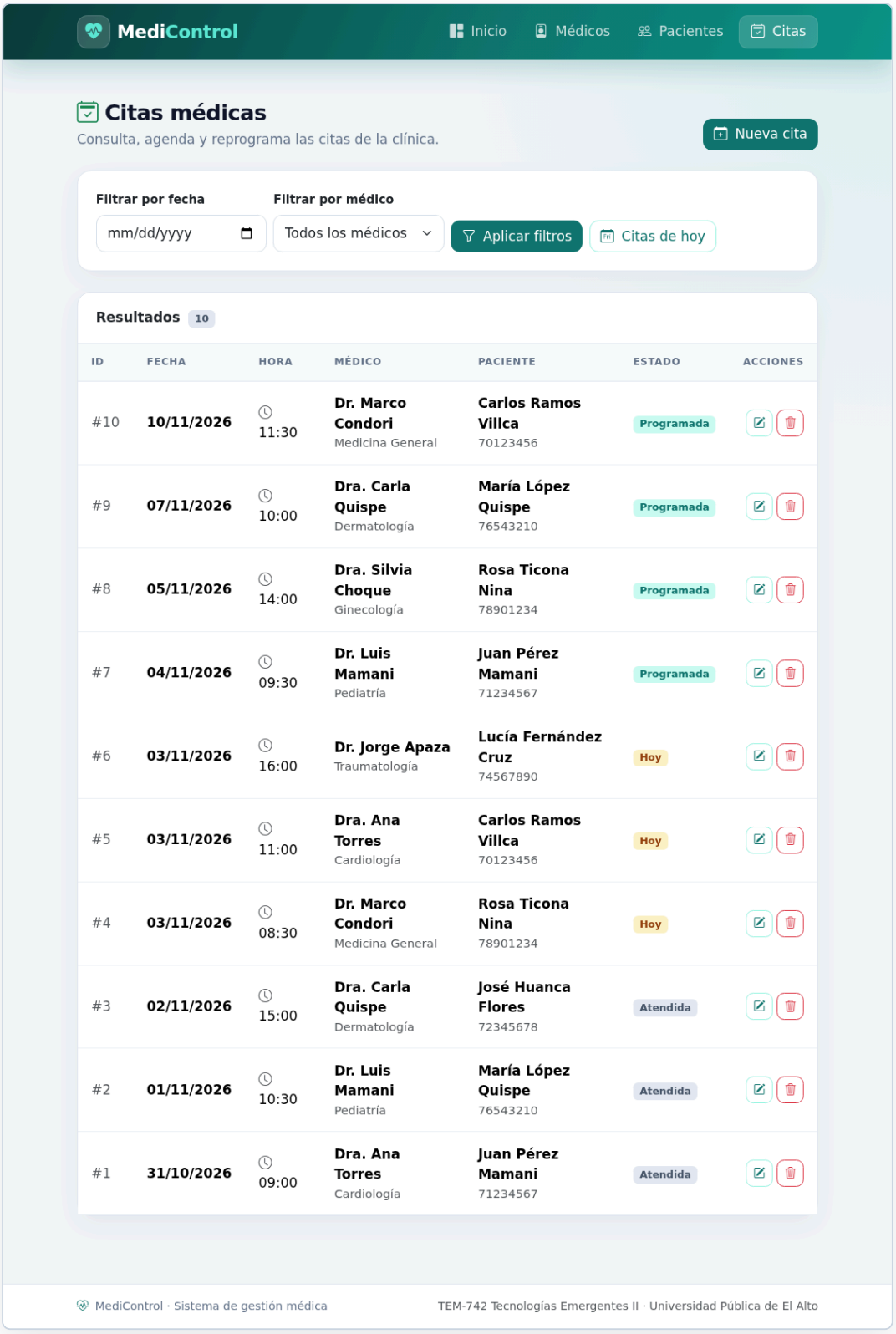


Figura 10. Listado de citas con 10 registros de prueba. Se muestran los filtros por fecha y médico, los estados y las acciones de edición y eliminación.

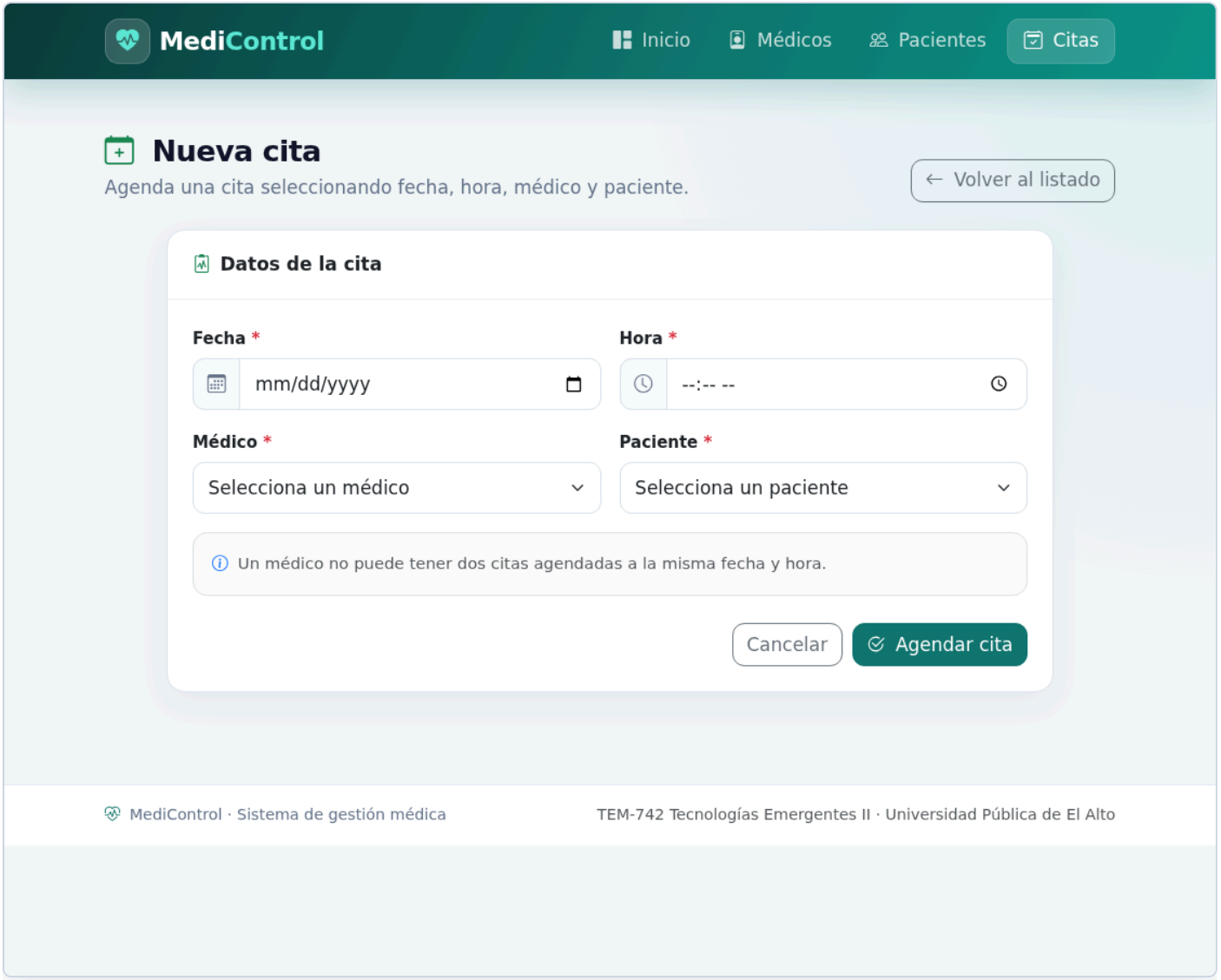


Figura 11. Formulario para agendar una cita. La fecha, la hora, el médico y el paciente son obligatorios; los médicos y pacientes se cargan dinámicamente desde la base de datos.

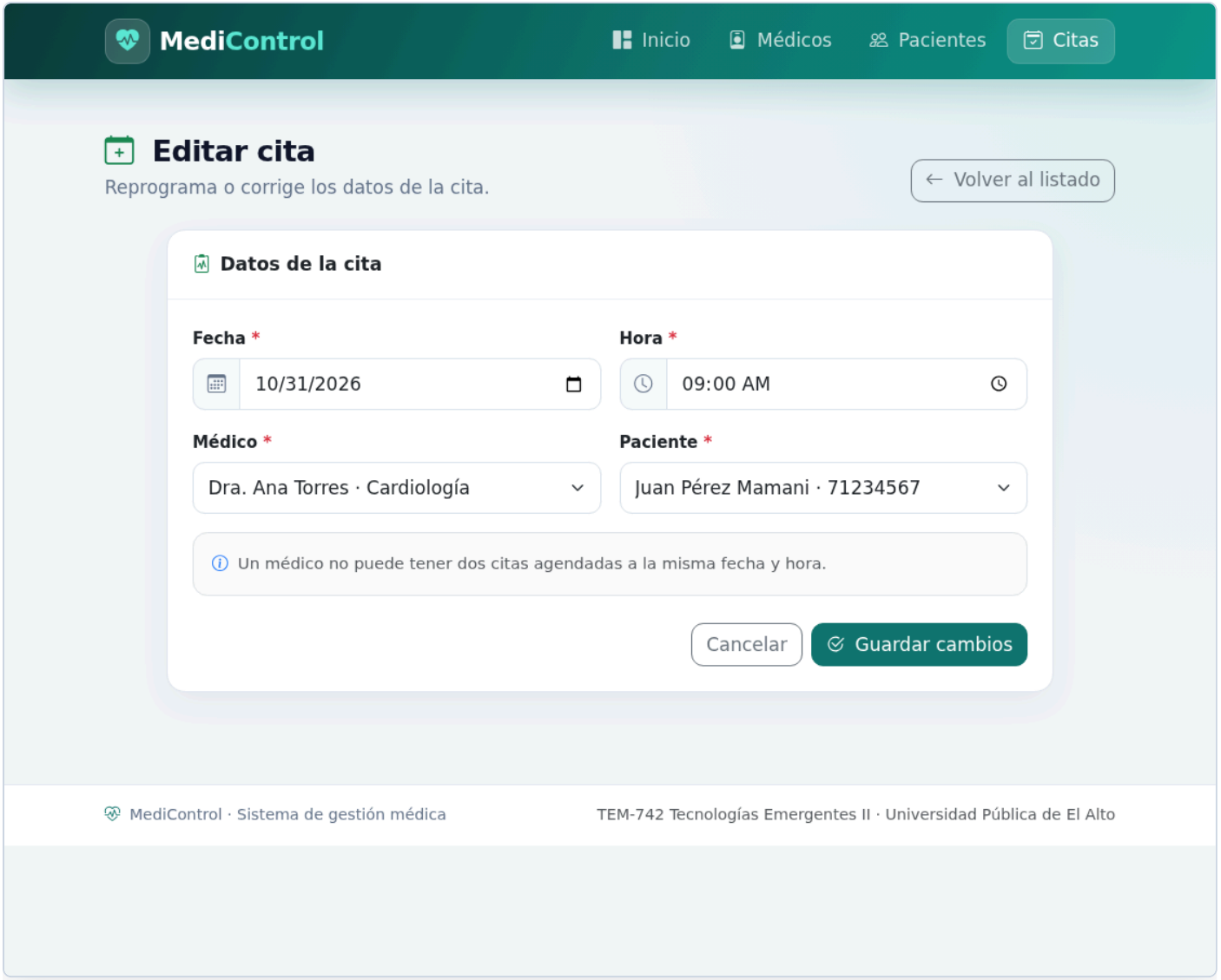


Figura 12. Edición de una cita existente. La misma plantilla se reutiliza para el alta y la modificación, conservando los valores seleccionados.

Resultado de la Parte 3. Los tres CRUD están completos y las plantillas heredan de `base.html`. La interfaz reutiliza componentes de Bootstrap, muestra mensajes flash y ofrece búsqueda, filtros y confirmaciones de borrado.

SECCIÓN 6 · PARTE 4

Funcionamiento de la aplicación

Carga de datos de prueba y ejecución de las operaciones más importantes del sistema.

6.1. Datos de prueba

El script `seed.py` inserta 6 médicos, 6 pacientes y 10 citas. Las fechas de las citas se calculan respecto al día de ejecución para que siempre existan citas pasadas, citas de hoy y citas futuras, lo que permite observar los tres estados del listado.

MÉDICOS	PACIENTES	CITAS
Dra. Ana Torres (Cardiología) Dr. Luis Mamani (Pediatría) Dra. Carla Quispe (Dermatología) Dr. Jorge Apaza (Traumatología) Dra. Silvia Choque (Ginecología) Dr. Marco Condori (Medicina General)	Juan Pérez Mamani María López Quispe José Huanca Flores Rosa Ticona Nina Carlos Ramos Villca Lucía Fernández Cruz	10 citas distribuidas entre 3 días pasados, el día actual y 7 días futuros, con distintos médicos, pacientes y horarios.



Figura 13. Ejecución de `seed.py --reset` y verificación de los registros insertados directamente en SQLite: 6 médicos, 6 pacientes y 10 citas.

6.2. Operaciones realizadas

Para comprobar el funcionamiento se realizaron las operaciones más importantes desde la interfaz web: alta de paciente, agendamiento de cita, intento de cita duplicada, reprogramación, cancelación de cita, baja de paciente e intento de baja de un médico con citas. Cada operación se describe a continuación con su captura.

1. Alta de un paciente

Se registró a la paciente **Valeria Quispe Rojas** con el teléfono **70998877**. Al guardar, el sistema redirige al listado y muestra el mensaje de confirmación. Este registro se utilizó después para crear una cita.

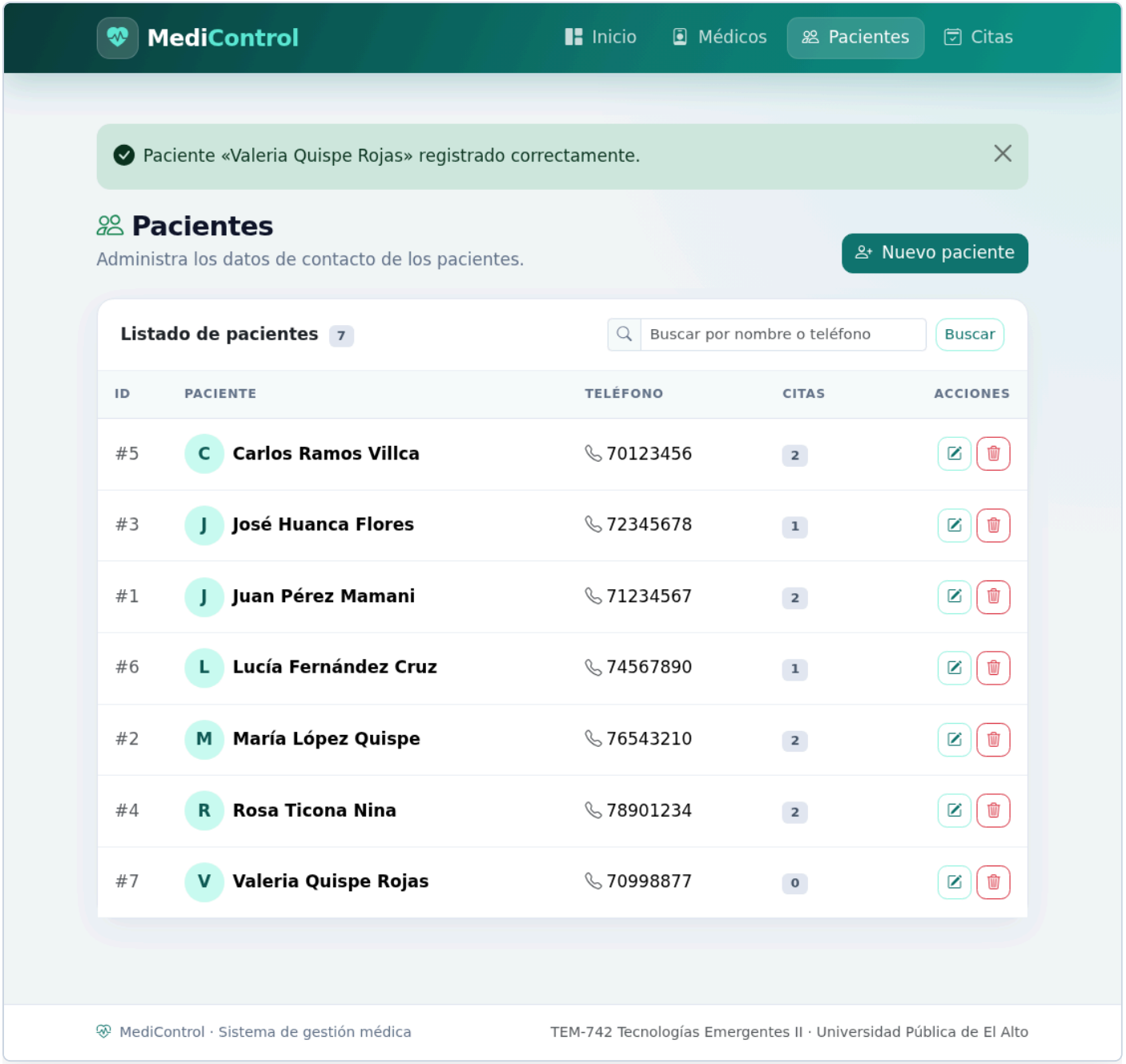


Figura 14. Confirmación del alta del paciente. El mensaje flash verde evidencia que la operación se completó y el nuevo registro aparece en el listado.

2. Agendamiento de una cita

Se seleccionó la fecha (tres días después de la fecha de ejecución), la hora 13:00, el médico **Dr. Marco Condori** y la paciente recién registrada. El formulario muestra las listas desplegables pobladas desde la base de datos.

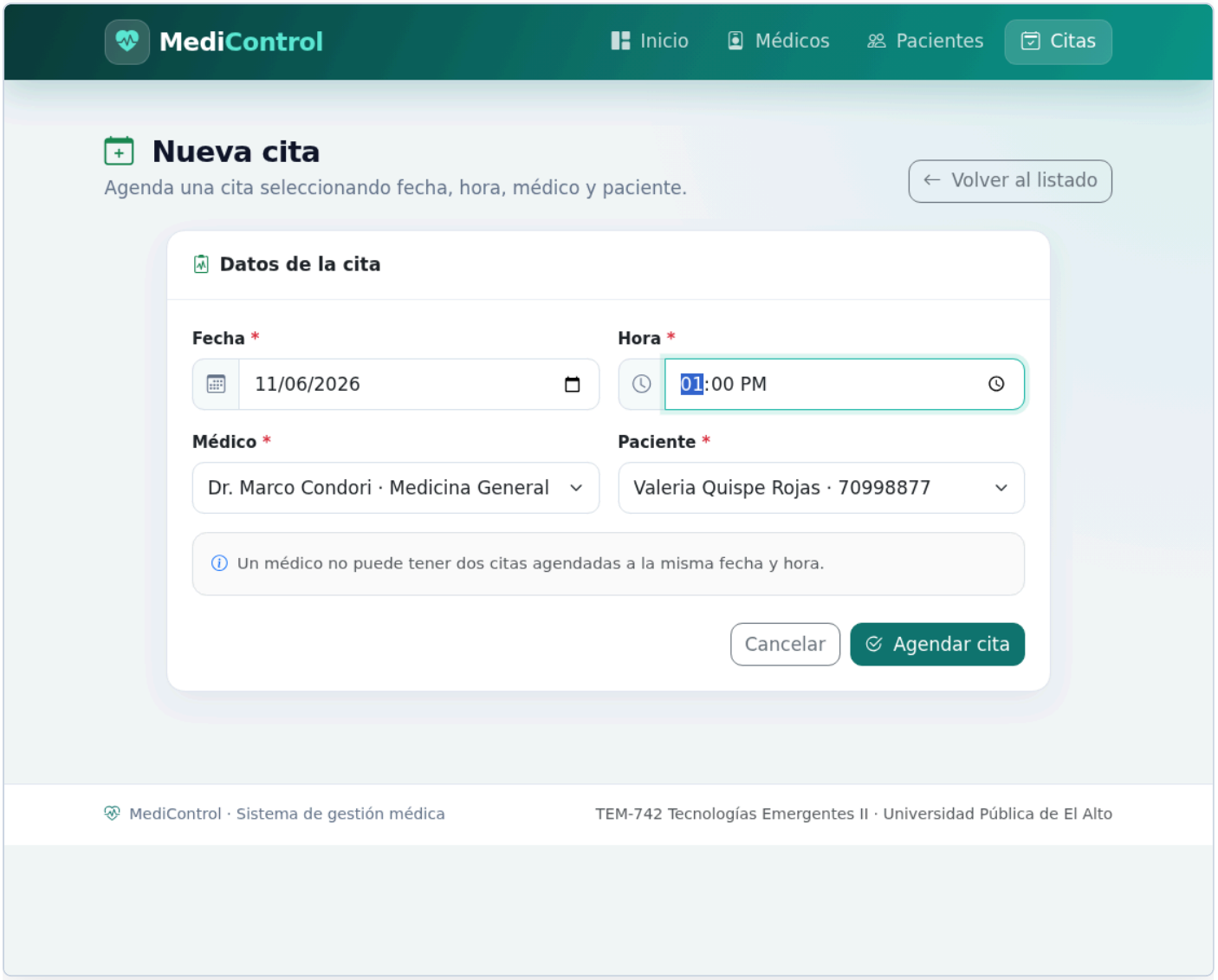


Figura 15. Formulario de cita con todos los datos seleccionados antes de guardar. La fecha y la hora se ingresan con controles nativos del navegador.

Al guardar, la cita se inserta y el listado muestra el mensaje de éxito. La nueva cita aparece como *Programada* y queda asociada al médico y a la paciente.

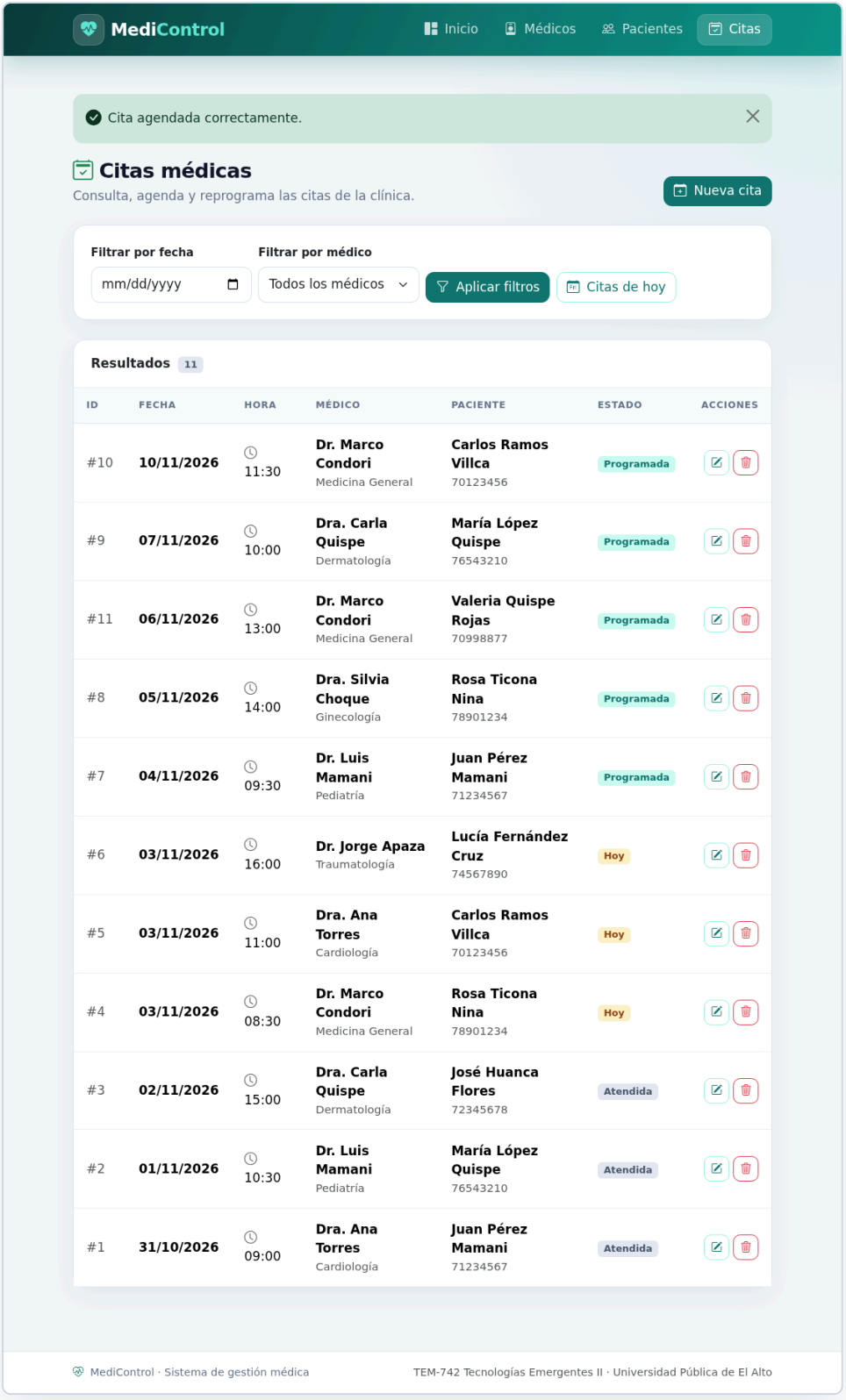


Figura 16. Cita agendada correctamente. El listado incorpora el nuevo registro y muestra la confirmación en la parte superior.

3. Validación de cita duplicada

Se intentó agendar otra cita para el mismo médico, la misma fecha y la misma hora. El sistema rechazó la operación con un mensaje de error y no insertó ningún registro. Esta validación se implementó en `app/blueprints/citas/routes.py`.

```
app/blueprints/citas/routes.py (fragmento)
```

```
consulta = Cita.query.filter_by(
    medico_id=medico.id, fecha=fecha, hora=hora
)
if cita_actual is not None:
    consulta = consulta.filter(Cita.id != cita_actual.id)

if consulta.first():
    errores.append(
        "El médico ya tiene una cita agendada en esa fecha y hora. "
        "Elige otro horario."
    )
```

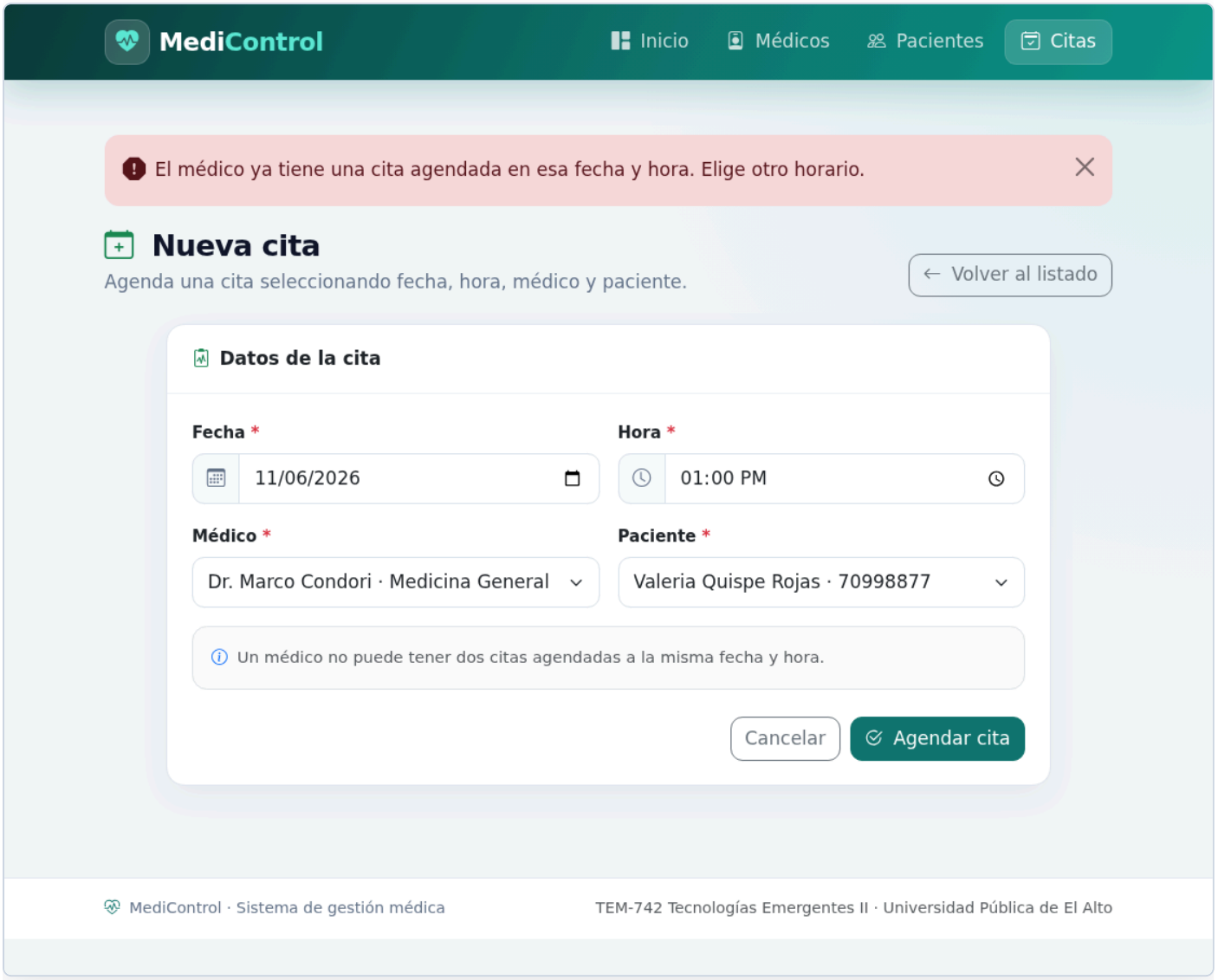


Figura 17. Rechazo de una cita duplicada. El formulario conserva los datos ingresados y muestra el mensaje de error; la base de datos no se modifica.

4. Reprogramación de la cita

Se editó la cita creada y se cambió la hora de 13:00 a 14:30. El sistema actualizó el registro y mostró la confirmación correspondiente.

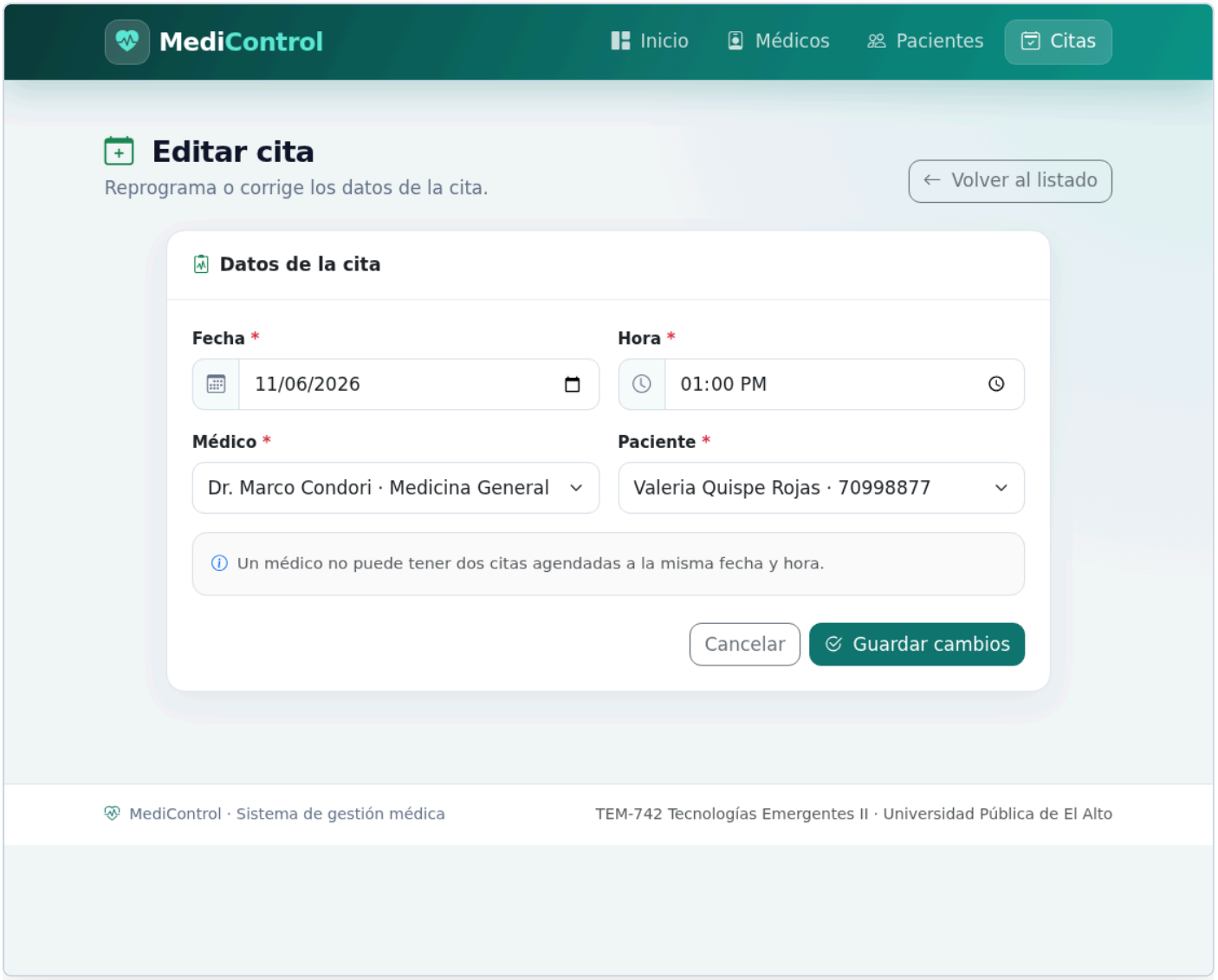


Figura 18. Formulario de edición con los datos actuales de la cita. Se modificó únicamente la hora para demostrar la actualización.

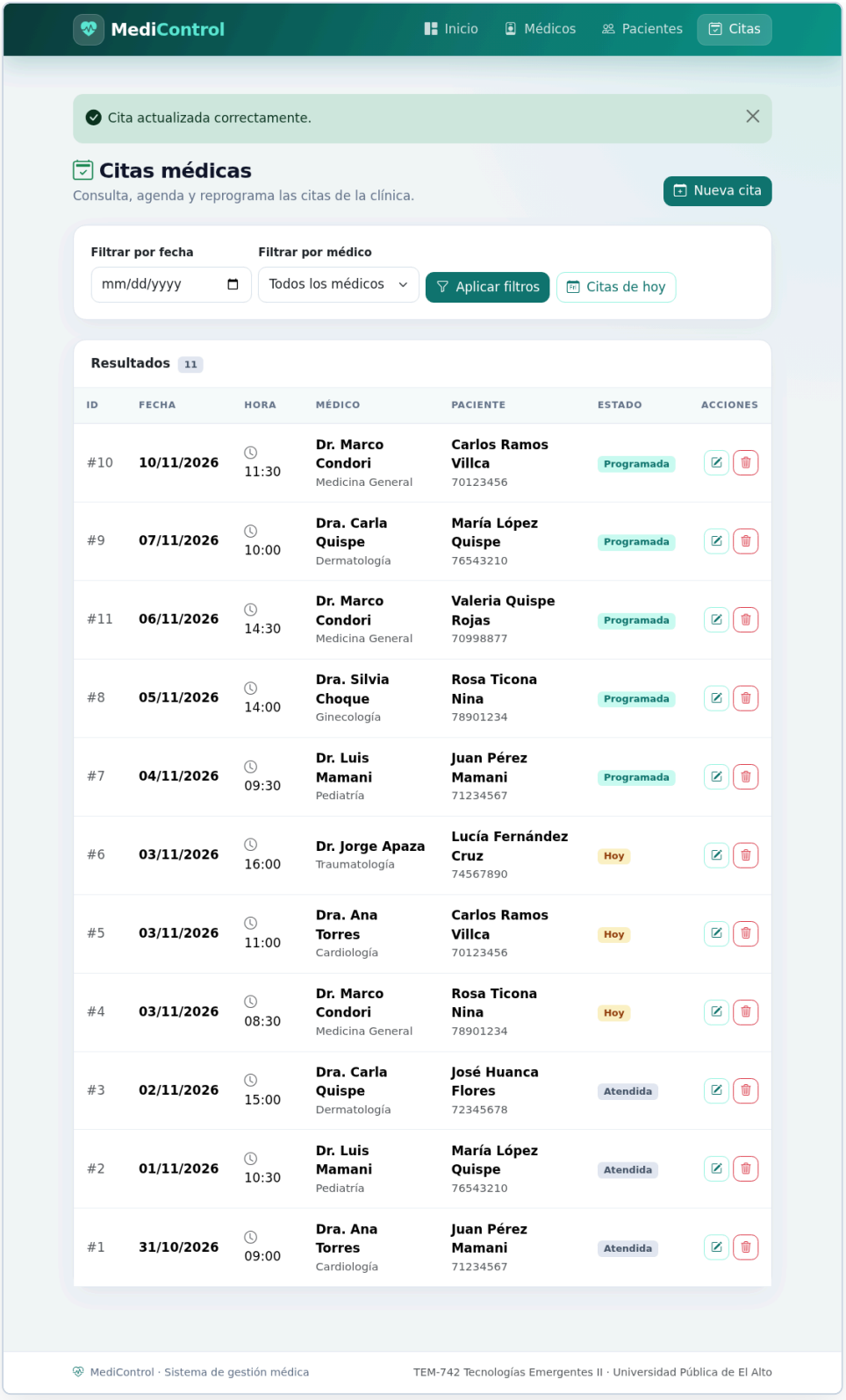


Figura 19. Cita reprogramada a las 14:30. El listado refleja el cambio y muestra el mensaje de actualización exitosa.

5. Cancelación de la cita

Se eliminó la cita creada para la demostración. La operación solicita confirmación y, al aceptarla, elimina el registro y muestra el mensaje de éxito.

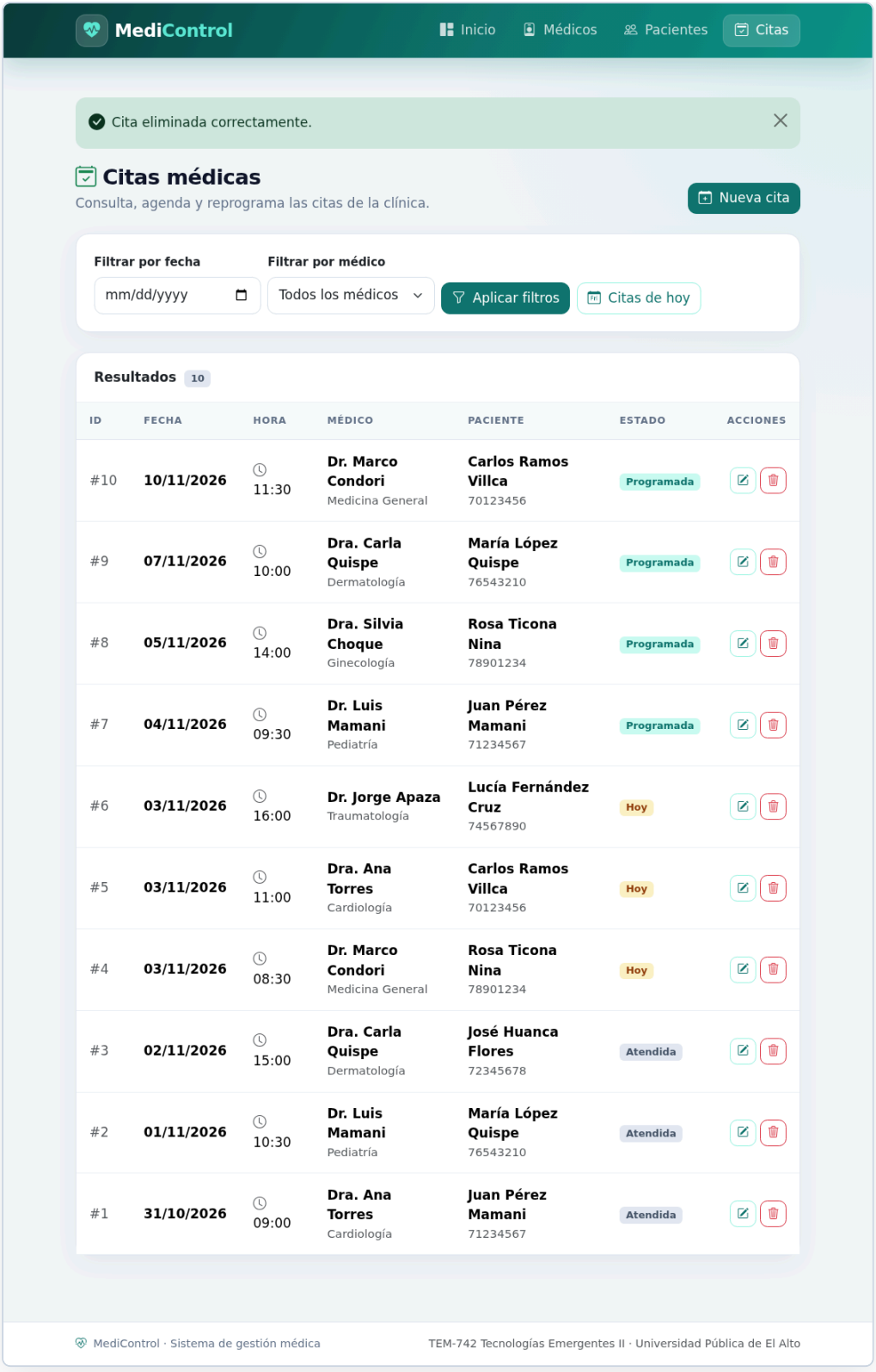


Figura 20. Cancelación de la cita. El registro desaparece del listado y se confirma la operación con un mensaje flash.

6. Baja del paciente de prueba

Una vez eliminada su cita, se dio de baja a la paciente de prueba. El sistema permitió la operación porque ya no tenía citas asociadas.

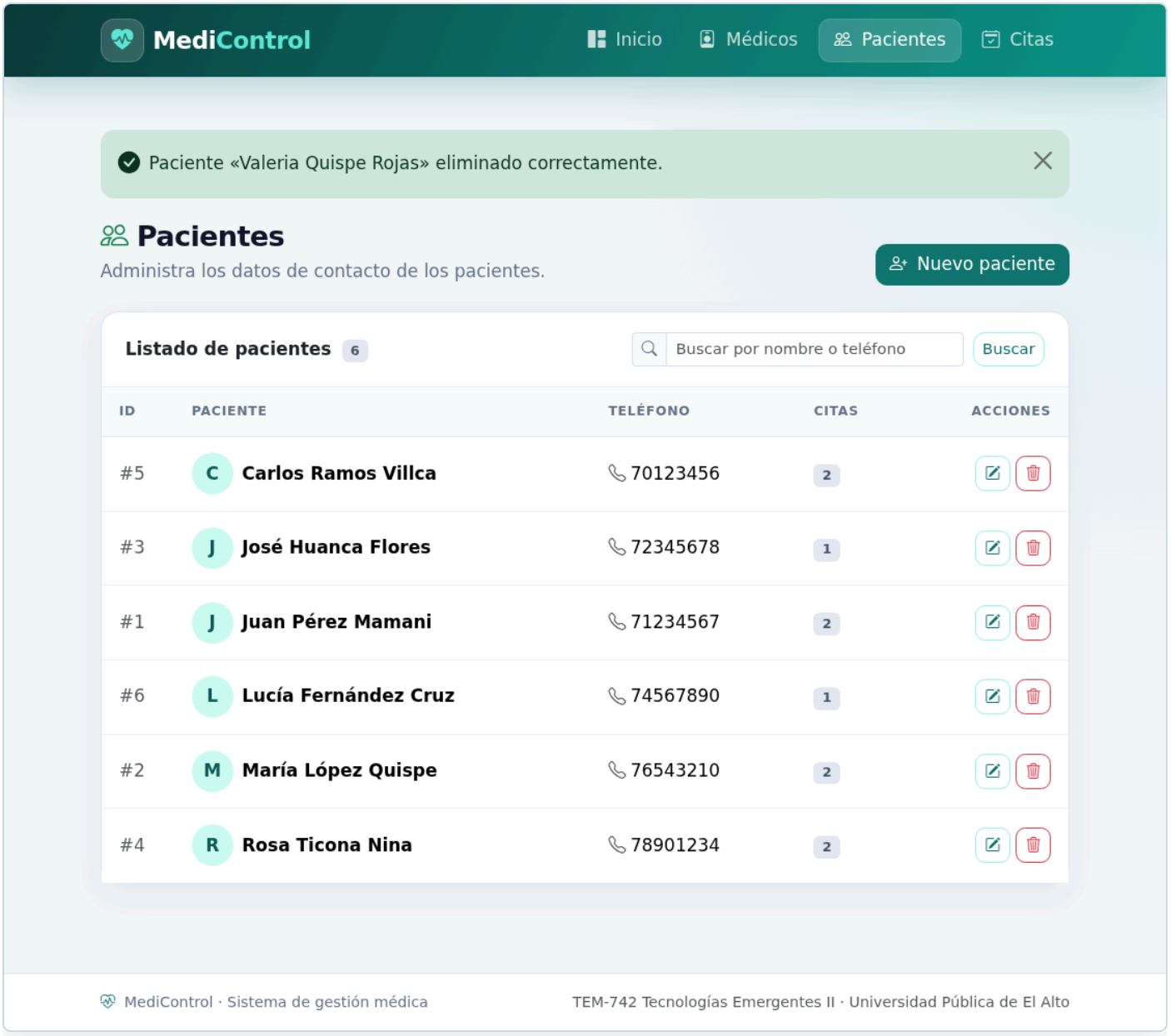


Figura 21. Baja del paciente de prueba. El listado vuelve a los 6 pacientes originales y muestra la confirmación.

7. Protección de la integridad referencial

Se intentó eliminar a la **Dra. Ana Torres**, quien tiene citas registradas. El sistema bloqueó la operación y mostró una advertencia indicando que primero deben eliminarse sus citas. La misma regla se aplica a los pacientes.

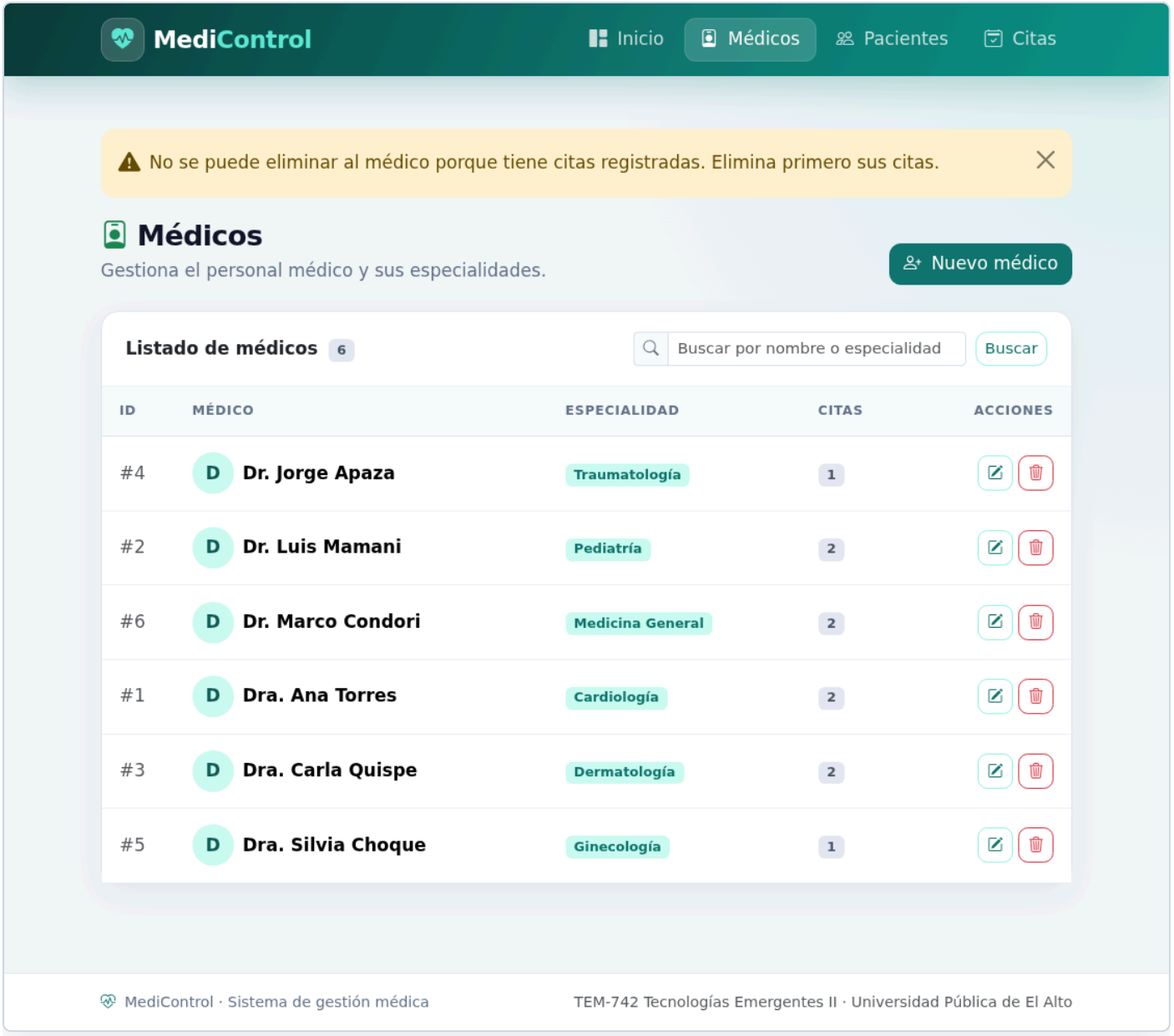


Figura 22. Intento de eliminar un médico con citas. El sistema conserva el registro y muestra una advertencia, evitando dejar citas huérfanas.

6.3. Reglas de validación comprobadas

REGLA	COMPORTAMIENTO OBSERVADO
Campos obligatorios vacíos	Se rechaza el formulario con mensajes de error y código HTTP 400.
Teléfono con formato inválido	Se valida con expresión regular y se muestra el formato permitido.
Cita duplicada (médico + fecha + hora)	Se rechaza la operación y no se inserta el registro.
Eliminar médico con citas	Se bloquea la baja y se muestra una advertencia.
Eliminar paciente con citas	Se bloquea la baja y se muestra una advertencia.
Filtros de citas por fecha o médico	Se muestran únicamente los registros que coinciden.

Resultado de la Parte 4. Las operaciones de alta, consulta, edición y baja funcionaron correctamente. Las validaciones protegieron la información y la base quedó nuevamente con los 6 médicos, 6 pacientes y 10 citas de prueba.

SECCIÓN 7

Conclusiones

El sistema **MediControl** cumple con todos los requerimientos del segundo parcial práctico. Se implementó una arquitectura monolítica modular con el patrón *Application Factory*, tres Blueprints independientes y una base de datos SQLite versionada mediante migraciones.

Logros técnicos

- Separación clara de responsabilidades entre configuración, modelos, rutas y plantillas.
- Relaciones uno a muchos correctamente declaradas y verificadas en la base de datos.
- Migraciones reproducibles: el esquema puede reconstruirse desde cero con un solo comando.
- CRUD completo para las tres entidades, con búsqueda, filtros, mensajes y confirmaciones.
- Validaciones de negocio que evitan duplicados y protegen la integridad referencial.
- Interfaz responsiva con Bootstrap 5 y recursos almacenados localmente.

Aprendizajes

El desarrollo permitió aplicar de forma integrada los temas de la materia: creación de aplicaciones con Flask, uso de Blueprints, mapeo objeto-relacional con SQLAlchemy, migraciones de esquema con Flask-Migrate, herencia de plantillas con Jinja2 y construcción de interfaces con Bootstrap. También se reforzó la importancia de validar los datos tanto en el cliente como en el servidor.

Trabajo futuro

Como mejoras futuras se podrían incorporar autenticación de usuarios, roles (administrador, recepcionista, médico), historiales clínicos, notificaciones por correo o WhatsApp, y una API REST para integrar el sistema con otras aplicaciones. La estructura modular actual facilita agregar estos módulos sin reescribir el código existente.

Mayte Mikaela Quispe Perez
Estudiante · TEM-742 Tecnologías Emergentes II

SECCIÓN 8

Anexo — Ejecución y entregables

8.1. Requisitos

- Python 3.13 o superior.
- Flask 3.1, Flask-SQLAlchemy 3.1, Flask-Migrate 4.1 y SQLAlchemy 2.1.
- SQLite (incluido con Python).
- Navegador web moderno.

8.2. Instalación y puesta en marcha

Terminal

```
# 1) Clonar el repositorio
git clone https://github.com/mikaelamay/segundo_parcial_mmqp.git
cd segundo_parcial_mmqp

# 2) Crear y activar el entorno virtual
python3 -m venv venv
source venv/bin/activate

# 3) Instalar dependencias
pip install -r requirements.txt

# 4) Aplicar las migraciones (crea instance/medicontrol.db)
flask --app run.py db upgrade

# 5) Cargar los datos de prueba
python seed.py

# 6) Ejecutar la aplicación
flask --app run.py run
```

Luego se abre `http://127.0.0.1:5000` en el navegador. La base de datos se crea automáticamente en `instance/medicontrol.db` y no se versiona en Git, por lo que debe generarse con las migraciones y el script de datos.

8.3. Repositorio del proyecto

ENTREGABLE 1 · URL DEL REPOSITORIO EN GITHUB

`https://github.com/mikaelamay/segundo_parcial_mmqp`

8.4. Entregables

✓ **Repositorio GitHub** con el código fuente completo, migraciones, datos de prueba y documentación.

- ✓ **Documento PDF** con identificación, capturas de pantalla de las cuatro partes y explicación breve de cada una.
- ✓ **README.md** con instrucciones de instalación, estructura del proyecto y rutas principales.

8.5. Estructura de la documentación

CARPETA / ARCHIVO	CONTENIDO
`app/`	Código de la aplicación: fábrica, modelos, Blueprints, plantillas y estáticos.
`migrations/`	Repositorio de migraciones y revisión inicial del esquema.
`instance/`	Base de datos SQLite generada localmente (excluida de Git).
`docs/`	Informe HTML, PDF final, capturas de pantalla y scripts de generación.
`seed.py`	Datos de prueba de médicos, pacientes y citas.
`config.py`	Configuración de la aplicación y cadena de conexión SQLite.
`run.py`	Punto de entrada para ejecutar el servidor Flask.

Nota final. Todas las capturas incluidas en este informe fueron tomadas de la aplicación real en ejecución, con los datos de prueba cargados. El proyecto puede reproducirse siguiendo los pasos del apartado 8.2.